

Security

UNIT 2

Lucas Bonanni

Security

Contenido:

- Concepts
- OWASP Top 10

Concepts

What is an API?

Application Programming Interface

Interfaz de software que permite a un software utilizar otro software.

API REST: la invocación (nombre del método y parámetros) se incrusta en una solicitud HTTP.

HTTP Request

Una API REST se invoca enviando una solicitud HTTP a la URL utilizando el protocolo especificado.

Estructura de una solicitud HTTP:

- Método (GET, POST, PUT, DELETE)
- URI (Uniform Resource Identifier)
- Versión de HTTP
- Encabezados (Headers)
- Cuerpo de la solicitud (Request Body)

Endpoint

Las API REST utilizan URI (Identificador Uniforme de Recursos) para identificar los recursos de una aplicación web.

Un endpoint es la URI que especifica el recurso

Puede considerarse como un nombre de método.

Estructura de una URL en una API REST:

Protocolo://Nombre de dominio/Versión/Endpoint

Vulnerability

Una vulnerabilidad es un comportamiento inesperado de un programa, que produce un efecto secundario en la confidencialidad, integridad o disponibilidad de la información que se pretende proteger.

El objetivo del atacante es encontrar comportamientos inesperados que el programador no previó.

Obtaining Unexpected Behaviors

Formas de obtener comportamientos inesperados:

- Valores inesperados para un parámetro de API dado
- Combinaciones inesperadas de valores para los parámetros de entrada
- Encadenamiento inesperado de invocaciones de endpoints
- Endpoints inesperados (ocultos)
- Parámetros de entrada inesperados (ocultos)

REST API Scenario

Escenario típico de una API REST mostrando la interacción entre cliente y servidor.

OWASP Top 10 - Evolución

Resumen de Años Anteriores (2019 y 2023)

Históricamente, OWASP ha publicado listas especializadas, como el **API Security Top 10** (2019 y 2023).

Evolución Principal (2019 a 2023): Se observó un claro cambio desde vulnerabilidades técnicas simples hacia fallas lógicas de negocio más complejas.

- **Autorizaciones Rotas:** BOLA (Broken Object Level Authorization) y BFLA (Function Level) se unificaron conceptualmente.
- **Exposición de Datos:** Se enfocaron más en la causa raíz (validación de propiedad y asignación masiva).
- **Nuevos Riesgos (2023):** Se introdujo el *Acceso no restringido a flujos de negocio* (ej. b... abusando de compras) y el *Consumo inseguro de APIs* (confiar en terceros).

OWASP Top 10 (2025)

Introducción al Top 10 2025

El OWASP Top 10 2025 representa el consenso actualizado sobre los riesgos de seguridad más críticos en aplicaciones web.

1. **A01** Broken Access Control
2. **A02** Security Misconfiguration
3. **A03** Software Supply Chain Failures
4. **A04** Cryptographic Failures
5. **A05** Injection
6. **A06** Insecure Design
7. **A07** Authentication Failures
8. **A08** Software or Data Integrity Failures
9. **A09** Security Logging and Alerting Failures
10. **A10** Mishandling of Exceptional Conditions

A01: Broken Access Control

Control de Acceso Roto

El control de acceso asegura que los usuarios no puedan actuar fuera de sus permisos previstos. Las fallas generalmente conducen a la divulgación, modificación o destrucción no autorizada de datos, o a la toma de control de funciones de administración.

A nivel técnico, esto ocurre cuando:

- Se manipulan tokens JWT, cookies de sesión o identificadores en la URL (IDOR/BOLA).
- Un endpoint diseñado para administradores no verifica correctamente el rol del usuario conectado.
- La aplicación confía en datos proveídos por el cliente en lugar del estado almacenado en el servidor.

A01: Empresas Afectadas

- **First American Financial (2019):** Sufrió una exposición masiva de 885 millones de registros financieros confidenciales (ataque IDOR), provocado por un control de acceso roto a nivel de objeto.
- **Uber (2019):** Investigadores de seguridad descubrieron críticas fallas BOLA en su API que exponían tokens de sesión y datos personales de conductores.

A01: Ejemplo de Explotación

“El Recibo Millonario”

Alice inicia sesión en el portal web de su banco. Al ver sus facturas, nota que la URL es `https://banco.com/recibos?id=80010`. Alice siente curiosidad: ¿Qué pasa si cambio el número final?

Al modificar el ID a `80011` en el navegador, el servidor verifica que Alice tiene una sesión iniciada válida, pero **omite verificar si el recibo 80011 le pertenece**. El sistema devuelve sin problemas el documento financiero del usuario Bob.

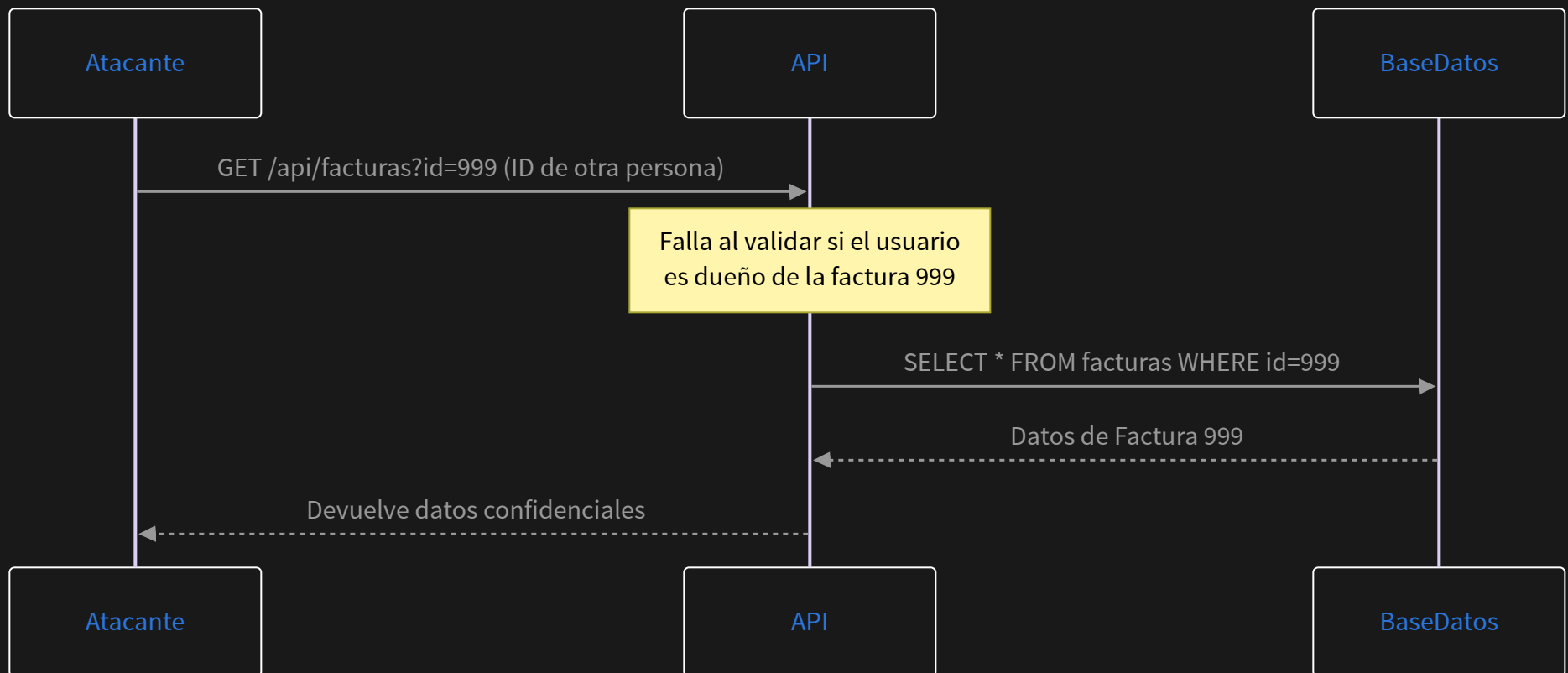
Alice utiliza un script de Python simple (`requests`) iterando automáticamente desde `id=1` hasta `id=1000000`. Extrae en horas el historial completo de cientos de miles de clientes sin disparar alarmas, convirtiendo un error diminuto de código en una brecha catastrófica.

A01: Contramedidas

¿Cómo mitigar el Control de Acceso Roto?

- **Denegar por defecto:** Todo acceso debe requerir autorización explícita, a menos que sea un recurso público.
- **Validar pertenencia (Ownership):** Verificar siempre en el backend que el usuario tiene permisos sobre el objeto específico solicitado (mitiga BOLA/IDOR).
- **Loggear fallos:** Registrar exhaustivamente todos los intentos fallidos de acceso a endpoints protegidos.
- **Deshabilitar listados:** Evitar listar directorios y reemplazar identificadores numéricos predecibles por UUIDs.
- **Establecer Rate limits:** Limitar la cantidad de veces que se puede llamar o consultar un recurso.

A01: Broken Access Control - Diagrama



A02: Security Misconfiguration

Configuración de Seguridad Incorrecta

Las aplicaciones, frameworks y servidores a menudo se despliegan de forma veloz con configuraciones inseguras de fábrica.

A nivel técnico, esto abarca: - **Cloud Computing:** Buckets de almacenamiento S3 sin políticas de restricción de lectura pública. - **Servidores web y DB:** Puertos expuestos directamente a internet o credenciales predeterminadas. - **Cabeceras HTTP:** Falta de directivas robustas de seguridad (como HSTS, Content-Security-Policy). - **Entornos:** Dejar el modo “Debug” o desarrollo activo en producción.

A02: Empresas Afectadas

- **Capital One (2019):** Un Web Application Firewall (WAF) mal configurado con permisos excesivos (IAM) en AWS permitió a una atacante sortear defensas y robar datos de 100 millones de clientes.
- **Twitch (2021):** Un error trivial humano durante un cambio de configuración en sus infraestructuras expuso a internet público todo su código fuente, repositorios internos y el historial de pagos.

A02: Ejemplo de Explotación

“El Servidor Silencioso”

Bob, un atacante oportunista, utiliza plataformas como **Shodan** para buscar servicios conocidos con configuraciones por defecto.

Identifica una IP de una empresa que responde abiertamente en el puerto 9200, característico de **Elasticsearch**. Bob utiliza el comando **curl** para consultar **http://[IP_OBJETIVO]:9200/_search?pretty**. El servidor devuelve inmediatamente volúmenes masivos de datos en formato JSON, ya que versiones antiguas no traían autenticación habilitada.

Sin necesidad de desarrollar complejos exploits, Bob extrae gigabytes de logs internos la empresa. Estos logs revelan tokens temporales, tráfico de usuarios y contraseñas de bases de datos impresas por error en el registro.

A02: Contramedidas

¿Cómo mitigar Configuraciones Inseguras?

- **Infraestructura como Código (IaC):** Automatizar, auditar y versionar las configuraciones en repositorios.
- **Principio de Menor Privilegio:** Eliminar características, servicios, y permisos innecesarios (buckets, puertos, roles).
- **Hardening estándar:** Aplicar guías de endurecimiento (CIS Benchmarks) a los entornos.
- **Parches automatizados:** Mantener actualizados frameworks y sistemas operativos para tapar vulnerabilidades.

A03: Software Supply Chain Failures

Fallas en la Cadena de Suministro de Software

Las aplicaciones modernas no se construyen desde cero; son ensamblajes que integran cientos de componentes open-source.

A nivel técnico, este riesgo se materializa cuando:

- Dependencias (**npm**, **PyPI**, **Maven**) comprometidas o contienen malware ofuscado.
- La infraestructura de integración continua (CI/CD) carece de controles de acceso fuertes.
- No se verifica la firma digital ni los hashes de integridad de las imágenes Docker que se descargan.

A03: Empresas Afectadas

- **SolarWinds (2020):** Miles de organizaciones y agencias gubernamentales instalaron una actualización de software oficial que había sido troyanizada internamente por espías.
- **Kaseya (2021):** Una vulnerabilidad en su software de gestión remota permitió a cibercriminales sortear antivirus y desplegar *ransomware* masivamente a sus clientes.

A03: Ejemplo de Explotación

“El Paquete Envenenado”

Una popular librería de Node.js, empleada para colorear texto en consola, fue abandonada por su mantenedor. Un grupo de atacantes logra transferir el control del paquete en GitHub presentándose como voluntarios mantenedores.

Lanzan la versión 1.2.5. El código fuente parece normal, pero al ejecutarse durante la compilación en el servidor CI/CD de una corporación, un script oculto entra en acción.

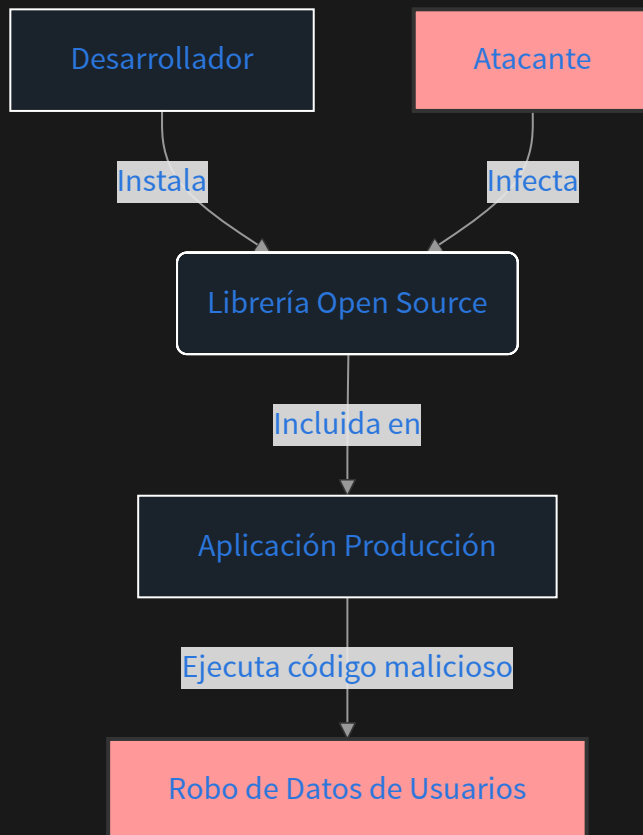
El código malicioso detecta el entorno de compilación, escanea las variables de sistema extrayendo tokens de AWS (`AWS_ACCESS_KEY_ID`) e IPs de bases de datos, y las transmite cifradas vía DNS hacia un servidor operado por los atacantes en otra jurisdicción.

A03: Contramedidas

¿Cómo mitigar Fallas en la Cadena de Suministro?

- **Verificar firmas y hashes:** Exigir sumas de comprobación antes de instalar librerías. Evitar usar **latest** en imágenes Docker.
- **Auditoría de Componentes (SCA):** Analizar vulnerabilidades conocidas en dependencias (generar y revisar el SBOM).
- **Zero Trust en CI/CD:** Asegurar el servidor de compilación aislando privilegios para que no comprometa a producción.
- **Repositorios Internos:** Almacenar copias auditadas de librerías locales, en lugar de descargar de internet continuamente.

A03: Software Supply Chain - Diagrama



A04: Cryptographic Failures

Fallas Criptográficas

Este riesgo engloba debilidades estructurales en la protección de datos valiosos almacenados (“en reposo”) o que viajan por la red (“en tránsito”).

A nivel técnico, se manifiesta mediante: - Uso de protocolos inseguros (HTTP en vez de HTTPS, FTP sin cifrar). - Hashing de contraseñas con algoritmos vulnerables o muy rápidos (MD5, SHA1) sin un valor aleatorio (“Salt”) anexado al hash. - Gestión inadecuada de llaves secretas (embebidas o “hardcodeadas” en el código). - Empleo del cifrado en modos de operación anticuados y previsibles (ej. modo ECB).

A04: Empresas Afectadas

- **Equifax (2017):** Hackeo que resultó en el robo de datos de 147 millones de ciudadanos. gran parte de los archivos residían en servidores completamente en texto plano sin cifrado alguno.
- **Adobe (2013):** Filtración de 153 millones de cuentas, cuyas contraseñas estaban cifradas usando un algoritmo reversible desactualizado sin “salt” en lugar de estar procesadas por hashes seguros.

A04: Ejemplo de Explotación

“El Rompecabezas del Criptoanálisis”

Un atacante se infiltra en una base de datos antigua. Nota que las contraseñas no están en texto plano, sino procesadas por el Hash **MD5**.

La aplicación no utiliza “Salting” (anexar caracteres aleatorios) y MD5 es un algoritmo diseñado para velocidad, no para proteger contraseñas.

El atacante alquila una granja de servidores con GPUs en la nube. Con software como *Hashcat*, la GPU genera **cien mil millones** de hashes por segundo. En un lapso de 12 horas, logra crackear el 75% de las contraseñas usando diccionarios masivos, y proceder a vender la información exponiendo a los usuarios en otras plataformas.

A04: Contramedidas

¿Cómo mitigar Fallas Criptográficas?

- **Cifrado en tránsito (TLS):** Forzar HTTPS estricto mediante HSTS y deshabilitar protocolos obsoletos.
- **Cifrado en reposo:** Proteger bases de datos y volúmenes con algoritmos estándar robustos (ej. AES-256).
- **Gestión de Secretos:** Nunca incrustar (“hardcodear”) llaves en código. Usar bóvedas de secretos (Vault, Secrets Manager).
- **Hashing fuerte:** Implementar funciones de hash de diseño lento (bcrypt, Argon2) y agregar un *Salt* robusto único a cada contraseña.

A05: Injection

Inyección

Se materializa cuando datos provenientes del usuario no son sanitizados ni validados antes de ser entregados a un motor que los ejecutará como comandos lógicos.

A nivel técnico sucede por: - Ensamblado manual de cadenas de texto para armar consultas de bases de datos (SQL) o comandos de terminal (OS Injection). - Falta de uso sistemático de consultas parametrizadas tipadas (*Prepared Statements*). - Afecta por igual a motores convencionales SQL, bases documentales (NoSQL Injection), herramientas ORM y consultas LDAP y XML.

A05: Empresas Afectadas

- **TalkTalk (2015):** Datos bancarios y personales de más de 150,000 abonados fueron robados mediante inyecciones simples tipo SQL en un portal web corporativo desatendido.
- **Mossack Fonseca (Panama Papers, 2016):** Esta gigantesca filtración a periodistas fue facilitada explotando puntos de inyección de código en una plataforma anticuada de despacho.

A05: Ejemplo de Explotación

“El Comentario Silencioso de Acceso”

El atacante ingresa a la antigua página de inicio de sesión de un panel administrativo corporativo. Sabe que la página arma sus verificaciones concatenando variables: `SELECT id, username FROM tabla_admin WHERE usuario = 'AQUI_INPUT' AND clave = 'AQUI_CLAVE'`

El atacante, sin conocer credenciales, ingresa en el campo Usuario: `admin' --`

Al ensamblarlo, el motor procesa lo siguiente: `SELECT id, username FROM tabla_admin WHERE usuario = 'admin' --' AND clave = '...'`

El operador `--` define el comienzo de un comentario lógico, provocando que ignore todo a su derecha. La verificación de contraseña desaparece del proceso y se concede acceso superusuario pleno al atacante instantáneamente.

A05: Contramedidas

¿Cómo mitigar Inyecciones?

- **Consultas Parametrizadas:** Separar los datos de las instrucciones (*Prepared Statements*) para que el intérprete nunca evalúe el *input* como código.
- **Uso de ORMs Seguros:** Emplear Mapeadores Objeto-Relacionales modernos que aplican parametrización por defecto.
- **Validación Positiva (Allow-listing):** Aceptar únicamente caracteres esperados (ej. números en un teléfono) en el lado del servidor.
- **Escapar variables dinámicas:** Si es imperativo concatenar, asegurar de codificar/escapar el contenido específico al motor destino.

A06: Insecure Design

Diseño Inseguro

El diseño inseguro no debe confundirse con una mala implementación: es la introducción sistemática de vulnerabilidades desde la fase conceptual de la arquitectura.

A nivel técnico, se describe por: - Desarrollar flujos lícitos pero abusables, donde la aplicación funciona tal cual el creador dictó, sin medir el impacto malicioso. - Omitir perfiles de modelado de riesgos, permitiendo transacciones sin limitaciones (*rate limit*). - Recuperaciones de contraseñas diseñadas para la comodidad extrema (preguntas burdas, pines cortos estáticos), sacrificando la seguridad.

A06: Empresas Afectadas

- **Zoom (2020):** El “Zoombombing” se debió a decisiones de diseño comercial: las reuniones no tenían contraseña por defecto y los ID de sesión eran numéricos cortos muy previsibles.
- **Tesla (2020):** La arquitectura conceptual del novedoso sistema “sin llave pasivo” permitía el robo de automóviles usando retransmisores caseros de radio.

A06: Ejemplo de Explotación

“El Bloqueo del Inventario Fantasma”

Una casa minorista lanza un producto altamente anticipado. En su diseño, el e-commerce estipula que al pulsar “Añadir al Carrito”, ese ítem físico queda reservado en bodega por 20 minutos.

Grupos maliciosos desarrollan avanzados *bots*. Al abrirse la venta, el script genera miles de sesiones anónimas concurrentes apretando en milisegundos “Añadir al Carrito” sobre todas las unidades existentes.

El código del retail realiza su labor excelentemente y reserva las unidades. Resultado: ningún cliente fidedigno logra comprar. Luego, los atacantes negocian liberar los carritos temporalmente a cambio de rescates; un claro caso donde un diseño omitió prevenir la saturación en inventarios limitados.

A06: Contramedidas

¿Cómo mitigar Diseños Inseguros?

- **Modelado de Amenazas:** Analizar riesgos (*Threat Modeling*) durante la arquitectura para prever el abuso lógico comercial.
- **Fricción por seguridad:** Añadir controles rítmicos (Rate Limiting) y verificaciones CAPTCHA en flujos valiosos.
- **Controles Lógicos de Negocio:** Requerir cuentas verificadas y limitar unidades por persona para evitar acaparamiento automático.
- **Seguridad por diseño (Shift-Left):** Involucrar a seguridad antes de escribir código para auditar casos de uso abusables.

A07: Authentication Failures

Fallas de Autenticación

Abarca la totalidad de debilidades sobre cómo las aplicaciones confirman y resguardan la identidad real de sus usuarios.

A nivel técnico y procesal se agrupa en:

- Ausencia de medidas de bloqueo temporal (*Account Lockouts*), dando rienda suelta a los ataques de fuerza bruta y *Credential Stuffing*.
- No requerir autenticación de múltiples factores (MFA/2FA) para transacciones confidenciales.
- Criterios de creación de contraseñas débiles.
- Procesos endebles en el manejo de sesiones: sesiones de larga vida o funciones de cierre de sesión que no invalidan internamente el token en el servidor central.

A07: Empresas Afectadas

- **Nintendo (2020):** Admitió que más de 300,000 perfiles valiosos de la red *Nintendo Network ID* fueron comprometidos, atribuido netamente al uso del ataque masivo de *Credential Stuffing*.
- **23andMe (2023):** Sufrió una extracción masiva de datos de genética privada de millones de personas conectadas entre sí, facilitada porque el MFA era opcional y los usuarios repetían contraseñas.

A07: Ejemplo de Explotación

“El Infiltrado del Reciclaje”

Eve adquiere en la *Dark Web* un mega-archivo comprimido con volcados de cientos de millones de correos y contraseñas extraídas de infracciones antiguas en foros.

Sabiendo que los seres humanos adoran la conveniencia de usar las mismas contraseñas para todo, Eve instala herramientas como *SentryMBA* para atacar ininterrumpidamente las páginas de inicio de sesión de prestigiosas billeteras digitales.

Las plataformas de pago atacadas no presentan un MFA general ni bloqueos precisos por reintento. De una prueba de 10 millones de correos, un 0.5% funciona perfectamente, otorgándole a Eve acceso legítimo en segundos a unas 50,000 cuentas de diferentes personas para extraer silenciosamente sus fondos.

A07: Contramedidas

¿Cómo mitigar Fallas de Autenticación?

- **MFA Obligatorio:** Implementar y exigir Múltiples Factores de Autenticación para acceder a recursos o funciones sensibles.
- **Defensas Anti-Fuerza Bruta:** Aplicar bloqueos de cuenta tras intentos repetidos y limitar las peticiones del endpoint.
- **Auditoría de Credenciales Expuestas:** Cotejar contraseñas nuevas en bases filtradas mundiales conocidas.
- **Gestión Estricta de Sesiones:** Invalidar tokens de cierre de sesión en el servidor y usar tiempos máximos absolutos de caducidad.

A08: Software or Data Integrity Failures

Fallas de Integridad de Software y Datos

Trata la peligrosa suposición y fe ciega hacia componentes y actualizaciones asumiendo que “por venir de la ruta normal” mantienen su integridad original.

A nivel técnico, abarca vulnerabilidades críticas como: - **Deserialización insegura:** Aceptar objetos lógicos codificados provistos por el usuario e intentar reconstruirlos nativamente en memoria, lo que resulta habitualmente en Ejecución Remota de Código (RCE). - Uso de redes de distribución de contenido (CDNs) sin aplicar mecanismos de Integridad de Subrecursos (SRI). - Falta de verificación estricta de firmas digitales de los parches de actualización al aplicarlos.

A08: Empresas Afectadas

- **Asus (2019):** En la masiva operación *ShadowHammer*, su herramienta oficial “ASUS Live Update” fue manipulada para entregar un troyano certificado con firmas criptográficas válidas a medio millón de usuarios.
- **CCleaner (2017):** Delincuentes superaron la seguridad en las redes corporativas y empalmaron secretamente malware militar directo al binario de distribución oficial antes de liberarlo al público.

A08: Ejemplo de Explotación

“El Disfraz de la Cookie Serializada”

Un desarrollador de una plataforma Java decide agilizar tiempos de respuesta guardando el “perfil del usuario” dentro de una Cookie local. Para mantener la estructura intacta, serializa un objeto Java codificado en Base64 y lo envía al navegador.

Un atacante visualiza la Cookie (`r00ABXNy...`) en las cabeceras HTTP y detecta que es Java nativo. A través de la herramienta automatizada `ysoserial`, arma un *Payload*: es un nuevo paquete que luce lícito pero en realidad es un código encapsulado de ejecución de comandos.

El atacante intercambia textualmente la Cookie normal por la forjada y recarga la web. El servidor web envía la cookie a la función `ObjectInputStream.readObject()`. Al reconstruirse la estructura en memoria RAM, el código malicioso ordena inmediatamente instalar una terminal oculta bidireccional (Reverse Shell), ganando control total del servidor en las sombras.

A08: Contramedidas

¿Cómo mitigar Fallas de Integridad?

- **Bloquear Deserialización Insegura:** Dejar de transmitir estados en objetos nativos serializados. Emplear formatos planos y estandarizados (JSON) combinados con validación estricta de esquemas.
- **Integridad de Subrecursos (SRI):** Añadir el hash de comprobación a *scripts* o estilos importados desde redes externas.
- **Firmar digitalmente:** Garantizar que en flujos de CI/CD todo artefacto (imágenes, código y dependencias) incluya firmas PGP que el entorno de producción valide antes de ejecutarlas.

A09: Security Logging and Alerting Failures

Fallas en Registro y Monitoreo de Seguridad

Sin un registro disciplinado (*Logging*) y paneles que alerten ante situaciones anómalas (*Monitoring*), las brechas actúan silentes y resultan imposibles de detectar y neutralizar.

A nivel técnico, es detectado frecuentemente por:

- Incidencias de alta gravedad o denegaciones persistentes de acceso no figuran en ninguna tabla.
- Se graban informes exclusivamente en medios locales dispersos, imposibilitando la aglomeración analítica con un motor integrado (SIEM) y perdiendo la prueba forense cuando la instancia se reinicia o borra.
- Los logs son escuetos y omiten detalles forenses críticos (Dirección IP origen, marcas temporales, usuario).
- Inexistencia total de programación de umbrales automáticos de contingencia y alertas urgentes a analistas.

A09: Empresas Afectadas

- **Marriott / Red Starwood (2018):** Atacantes sofisticados operaron clandestinamente dentro de su red, apropiándose pasaporte y pagos de medio billón de hospedados durante **cuatro extensos años**.
- **Equifax (2017):** El hurto ininterrumpido de archivos financieros aconteció por **76 días de letargo**, provocado porque un sistema de alerta paralizó su análisis al omitirse actualizar su certificado criptográfico en fecha.

A09: Ejemplo de Explotación

“El Descenso de la Madrugada”

A través de la ingeniería social (phishing), un atacante doblega el ordenador de soporte de una corporación. Aprovechando fallas de arquitectura, escala obteniendo perfil de solo lectura en la base principal de clientes.

Para no congestionar las líneas diurnas ni levantar sospechas, el sujeto programa silenciosamente rutinas internas que, cada noche a las 3:00 de la madrugada los fines de semana, empaquetan listados masivos de la base, canalizándolos encriptados a servidores en jurisdicción extranjera.

La empresa carecía del desarrollo de visibilidad telemétrica ni monitoreos proactivos a transferencias externas masivas (“Egreso Clandestino”). La inacción al no evaluar a tiempo alertas de “sesiones anómalas de la cuenta directiva de madrugada”, brindan una vía franca por diez meses consecutivos para sustraer información valiosa antes de exigir rescate.

A09: Contramedidas

¿Cómo mitigar Fallas en Logging y Monitoreo?

- **Registrar acciones críticas:** Loggear inicios de sesión fallidos/exitosos, cambios de permisos e interacciones con bases confidenciales.
- **Centralización (SIEM):** Exportar los logs continuamente en tiempo real a una plataforma central externa y resguardada para evitar su borrado local.
- **Calidad de metadatos:** Enlazar siempre marca de tiempo (UTC), IP, usuario involucrado y estado de la solicitud.
- **Alertas Proactivas:** Programar alertas a los canales del equipo técnico si los umbrales normales estadísticos (anomalías volumétricas) son excedidos.

A10: Mishandling of Exceptional Conditions

Mal Manejo de Condiciones Excepcionales

Instaurado formalmente a partir del Top 10 de 2025 en OWASP, relata todas las exposiciones catastróficas del accionar lógico ante escenarios de fallos técnicos excepcionales.

A nivel técnico se evidencia por:

- Mostrar en texto abierto la traza de fallos de servidor (*Stack Traces* técnicos completos) hacia el internet público bajo contingencias operacionales.
- Aplicar un enfoque de “Apertura forzada ante fallos” (*Fail-open*). En el escenario donde la validación defensiva estricta de una regla falle por *timeout*, permite el acceso asumiendo que “todo está bien”.
- Tolerancia débil a volúmenes de parámetros no sanitizados desbordantes, conduciendo al colapso absoluto (Crash) y denegación de servicio.

A10: Empresas Afectadas

- **Cloudflare (2017 - Fenómeno “Cloudbleed”)**: Una falla severa de sintaxis provocó un quiebre de los *buffers*, procediendo inadvertidamente en verter por un lapso extenso contenido de memoria temporal cruzado (contraseñas e identificaciones de clientes de otros servidores) hacia dominios públicos HTTP fallidos.
- **T-Mobile (2021)**: Recibió críticas porque APIs de sus subdominios colapsaban, imprimiendo el despiece del ruteo, configuraciones precisas corporativas y detalles estructurales backend que otorgaron a los atacantes coordinadas logísticas infalibles.

A10: Ejemplo de Explotación

“El Mapa en la Explosión”

Motivado en descubrir brechas en un portal corporativo seguro, un criminal focaliza sus energías analizando el punto lícito donde la aplicación procesa la búsqueda rápida del legajo: `.../api/finder?profile_name=Juan`.

Sopesando que la consulta normal no surte anomalías, transmite metódicamente cadenas gigantescas mezclando notaciones matemáticas extrañas: `.../api/finder?profile_name=$$[{"'\\"@`. El gestor servidor (entorno Python) asume una aserción inaudita, escapándose radicalmente de los bloques previstos del `try-catch`, y fracturando dramáticamente el código intermedio.

El servidor, en lugar de despachar un cabezal de error mudo (`Error General 500 Interno`), responde en pantalla con la interfaz de diagnóstico y modo Debug en plena producción. El hacker no solo nota el percance; se encuentra con la ruta precisa del archivo de origen (`/var/app/handlers/sql_parser.py`), las variables temporales locales en memoria y la versión exacta de base de datos. Acaba de recibir en segundos anatomía, cartografía y esqueleto del backend sin ni siquiera sortear murallas defensivas de red.

A10: Contramedidas

¿Cómo mitigar el Mal Manejo de Excepciones?

- **Mensajes de Error Genéricos:** Jamás mostrar detalles técnicos (“Stack Traces”) a los usuarios finales en producción. Devolver mensajes formales (“Ha ocurrido un error inesperado con ID X”).
- **Aislamiento Técnico:** Registrar la anatomía detallada de la excepción internamente lado backend, para resolver el bug en la oscuridad, enlazado al ID público.
- **Fail-Secure:** Si una validación perimetral web falla estrepitosamente o por carencia recursos, el sistema debe bloquear o negar, nunca saltarla “Fail-Open”.
- **Limitación Absoluta:** Fijar cortes tajantes a las cadenas masivas defectuosas y configuraciones severas de límite de tamaño (*payload limit*) para no ahogar al intérprete.

Clase Siguiente

Próximos temas...